

一.引入信号

1.信号

生活中可以通过红绿灯信号控制车辆的行走

红灯 --》 停车

绿灯 --》 行车

linux中也有类似的概念，也能通过linux定义好的信号控制进程的执行

发送信号的shell命令

kill 信号的名字/信号序号 进程的id号

killall 信号的名字/信号序号 进程的名字

比如： kill -SIGINT 进程id号

kill -INT 进程id号

kill -2 进程id号

killall -SIGINT 进程的名字

二.linux中定义了哪些信号

```
root@ubuntu:~# kill -l
1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL      5) SIGTRAP
6) SIGABRT     7) SIGBUS     8) SIGFPE      9) SIGKILL     10) SIGUSR1
11) SIGSEGV    12) SIGUSR2    13) SIGPIPE    14) SIGALRM     15) SIGTERM
16) SIGSTKFLT  17) SIGCHLD    18) SIGCONT    19) SIGSTOP     20) SIGTSTP
21) SIGTTIN    22) SIGTTOU    23) SIGURG     24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM  27) SIGPROF    28) SIGWINCH   29) SIGIO        30) SIGPWR
31) SIGSYS     34) SIGRTMIN   35) SIGRTMIN+1 36) SIGRTMIN+2 37) SIGRTMIN+3
38) SIGRTMIN+4 39) SIGRTMIN+5 40) SIGRTMIN+6 41) SIGRTMIN+7 42) SIGRTMIN+8
43) SIGRTMIN+9 44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
58) SIGRTMAX-6  59) SIGRTMAX-5  60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1  64) SIGRTMAX
```

重要常用的信号

19) SIGSTOP --》 让进程暂停执行

18) SIGCONT --》 让进程继续执行

2) SIGINT --》 键盘输入ctrl+c默认就是给当前进程发送了SIGINT信号

9) SIGKILL --》 把进程杀死

三.linux中提供跟信号有关的接口函数

1.发送信号

```
int kill(pid_t pid, int sig);
```

返回值: 成功 0 失败 -1

参数: pid --》 你要发送信号的那个进程的id号

sig --》 你要发送哪个信号(信号的名字或者序号都行)

2.捕捉信号并改变信号的响应动作

```
void (*signal(int sig, void (*func)(int)))(int);
```

总结：返回值类型 (*函数名(形参))(函数指针对应的形参)

返回值：也是一个函数指针void()(int)

参数：sig --》你要捕捉的信号

void (*func)(int) --》你捕捉到该信号以后，需要改变的动作

int的参数用来存放你捕捉到的信号序号

signal函数的三种用法

第一种：改变信号的响应

signal(信号,函数指针) 当进程收到指定的信号之后,会自动调用函数指针指向的函数来执行响应

动作

第二种：忽略信号--》当外界给进程发送这个信号的时候，进程选择左耳进右耳出

signal(信号,SIG_IGN)

第三种：恢复默认动作

signal(信号,SIG_DFL)

3.阻塞进程，等待信号的到来

int pause(void);

4.阻塞信号(屏蔽信号)

阻塞信号代码编写的思路：

第一步：定义变量，保存所有你想要阻塞的信号

sigset_t类型的变量(本质上是数组) --》linux中专门用来保存阻塞信号，称之为信号阻塞掩码集

int sigemptyset(sigset_t *set); //清空集合

int sigfillset(sigset_t *set); //把linux中62个信号一起添加到set集合中

int sigaddset(sigset_t *set, int signum); //把signum信号添加到set集合中

int sigdelset(sigset_t *set, int signum); //把signum信号从集合中剔除

int sigismember(const sigset_t *set, int signum); //判断signum在不在set集合中，在集合中返回1，不在0

第二步：设置信号的阻塞

int sigprocmask(int how, const sigset_t *set,sigset_t *oset);

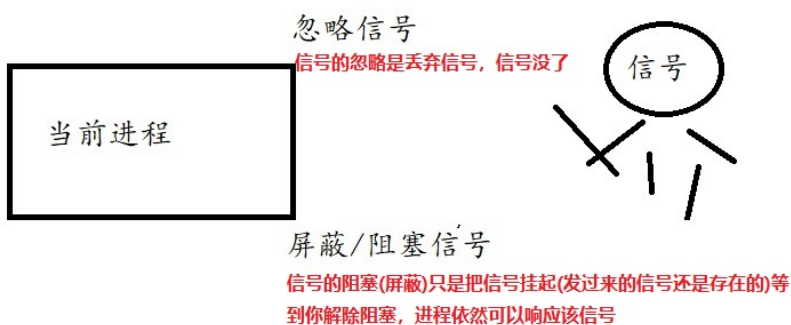
参数：how --》SIG_BLOCK //设置信号阻塞，把set和oset两个集合中的所有信号都阻塞

SIG_UNBLOCK //解除信号阻塞

set --》保存你想要阻塞的所有信号

oset --》备份，保存之前你设置的阻塞信号

阻塞信号和忽略信号的区别：



四.linux中信号四种响应方式

第一种：按照信号的默认动作响应

linux已经规定好了所有信号的默认动作，绝大部分默认动作都是终止进程

信号	值	缺省动作	备注
SIGHUP	1	终止	控制终端被关闭时产生
SIGINT	2	终止	从键盘按键产生的中断信号（比如 Ctrl+c）
SIGQUIT	3	终止并产生转储文件	从键盘按键产生的退出信号（比如 Ctrl+\）
SIGILL	4	终止并产生转储文件	执行非法指令时产生
SIGTRAP	5	终止并产生转储文件	遇到进程断点时产生
SIGABRT	6	终止并产生转储文件	调用系统函数 <code>abort()</code> 是产生
SIGBUS	7	终止并产生转储文件	总线错误时产生
SIGFPE	8	终止并产生转储文件	处理器出现浮点运算错误时产生
SIGKILL	9	终止	系统杀戮信号
SIGUSR1	10	终止	用户自定义信号
SIGSEGV	11	终止并产生转储文件	访问非法内存时产生
SIGUSR2	12	终止	用户自定义信号
SIGPIPE	13	终止	向无读者的管道输入数据时产生
SIGALRM	14	终止	定时器到点儿时产生
SIGTERM	15	终止	系统终止信号
SIGSTKFLT	16	终止	已废弃
SIGCHLD	17	忽略	子进程暂停或终止时产生
SIGCONT	18	恢复运行	系统恢复运行信号
SIGSTOP	19	暂停	系统暂停信号
SIGTSTP	20	暂停	由控制终端发起的暂停信号
SIGTTIN	21	暂停	后台进程发起输入请求时控制终端产生该信号
SIGTTOU	22	暂停	后台进程发起输出请求时控制终端产生该信号
SIGURG	23	忽略	套接字上出现紧急数据是产生
SIGXCPU	24	终止并产生转储文件	处理器占用时间超出限制值时产生
SIGXFSZ	25	终止并产生转储文件	文件尺寸超出限制值时产生
SIGVTALRM	26	终止	由虚拟定时器产生
SIGPROF	27	终止	<code>profiling</code> 定时器到点儿时产生
SIGWINCH	28	忽略	窗口大小变更时产生
SIGIO	29	终止	I/O 变得可用时产生
SIGPWR	30	终止	启动失败时产生
SIGUNUSED	31	终止并产生转储文件	同 SIGSYS

第二种：改变信号的响应动作

signal函数去捕捉信号并改变

注意：有两个信号SIGKILL和SIGSTOP很特殊，不能改变它们的默认动作

第三种：忽略信号

左耳进右耳出，当信号发送过来的时候，进程当它不存在，直接舍弃该信号(不存在了)

注意：有两个信号SIGKILL和SIGSTOP很特殊，不能忽略

第四种：信号的阻塞(信号的屏蔽)

阻塞信号仅仅只是把信号挡在进程的外面(暂时挂起), 不去响应, 等到解除阻塞, 依然能够正常响应信号(信号依然存在)

注意: 有两个信号SIGKILL和SIGSTOP很特殊, 不能阻塞

五.进程的状态切换(生老病死)

1.进程的六种状态和相互切换条件

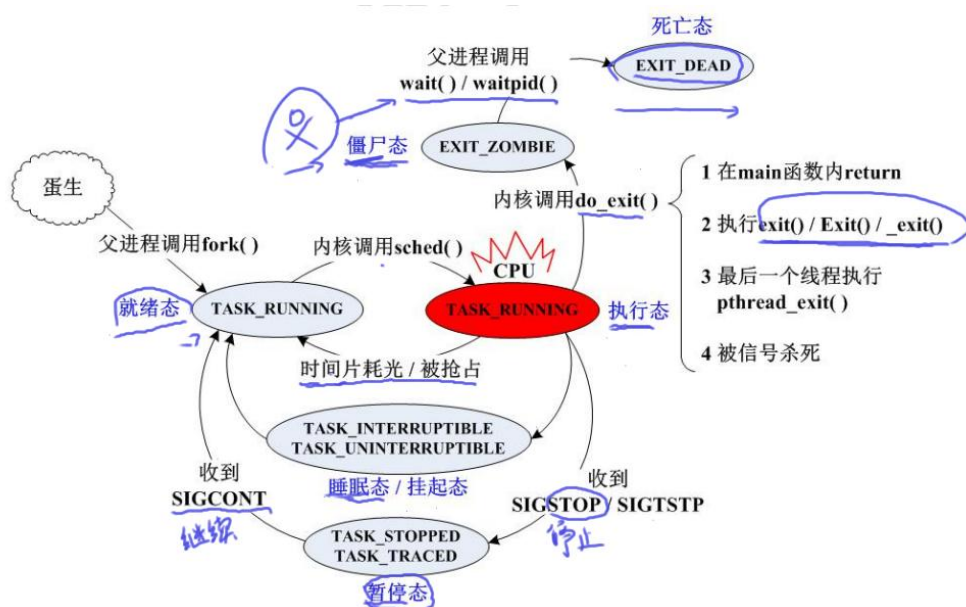


图 5.2 Linux 进程状态转换图

就绪态: 进程已经运行了, 但是还没有获取cpu的使用权, 当cpu的调度算法切换到该进程, 那么进程就能转入到执行态

执行态: 进程已经运行, 并且cpu的使用权也获取了

睡眠态/挂起态: 使用了sleep()--》进程处于睡眠态

带有阻塞功能的函数(pause() wait() waitpid() read), 进程都会进入挂起态(阻塞态)

暂停态: 进程收到暂停信号

僵尸态: 子进程退出了, 但是父进程没有调用wait/waitpid函数去回收,子进程就变成僵尸态

死亡态: 进程退出了, 有回收

六.练习作业

1.p1进程把自己的id号通过有名管道发送给p2进程, p2进程使用kill函数发送信号控制p1暂停, 继续, 退出

2.给进程发送SIGKILL和SIGSTOP,要求该进程捕捉这两个信号并改变它们的行为动作

3.用3) SIGQUIT 4) SIGILL 5) SIGTRAP这三个信号分别代表红绿黄交通信号灯

要求: 当进程收到红灯信号SIGQUIT, 程序打印“红灯停”

当进程收到绿灯信号SIGILL, 程序打印“绿灯行”

当进程收到绿灯信号SIGTRAP, 程序打印“黄灯注意观察”

4.用信号模拟司机售票员: 创建子进程代表售票员, 父进程代表司机,

1: 售票员捕捉SIGINT(代表开车), 发SIGUSR1 给司机, 司机捕捉到SIGUSR1 信号后打印“move to next station”

2: 售票员捕捉SIGQUIT(代表靠站), 发SIGUSR2 给司机, 司机捕捉到SIGUSR2 信号后打印“stop the bus”

3: 司机捕获到SIGTSTP（车到总站），发SIGUSR1给售票员，售票员捕捉到SIGUSR1信号后打印“all get off the bus”，全部进程结束

5.编写一段程序，创建两个子进程，用signal让父进程捕捉SIGINT,当捕获到SIGINT,父进程用kill向两个子进程发信号SIGUSR1/SIGUSR2

当两个子进程收到发来的信号后，分别输出以下信息及信号的标号后终止

Child process 1 is killed my parent: 信号标号

Child process 2 is killed my parent: 信号标号

父进程等待两个子进程终止后，输出以下信息后终止

Parent process exit;